

Contents

Hinweise, Taktik & Lösungsgedanken zu den Backlog-Storys	1
Erst mal die Karte verstehen	1
Epic 1 — Bewegung & Überleben	2
Epic 2 — Angriff	5
Epic 3 — Strategie & Zustände	7
Epic 4 — Qualität (optional)	9
Epic 5 — Turniervorbereitung	10
Wenn ihr komplett feststeckt	10

Hinweise, Taktik & Lösungsgedanken zu den Backlog-Storys

Diese Datei ist eure **Denkhilfe** für die Aufgaben aus [backlog.md](#). Sie gibt **keine fertige Lösung** vor, sondern hilft euch, selbst auf den Lösungsweg zu kommen: mit Bildern, Beispielen und Leitfragen.

Lest vorher unbedingt einmal den [Framework-Guide](#) — dort steht, was sensors enthält und welche Actions es gibt. Wenn ihr an einer Story hängt, sucht sie hier und arbeitet die Leitfragen der Reihe nach ab.

Erst mal die Karte verstehen

Alles dreht sich um Positionen auf einem 10×10-Raster. Wer die Koordinaten nicht sicher im Kopf hat, verrechnet sich bei **jeder** Story. Also zuerst das hier verinnerlichen:

Die drei Dinge, die euch am häufigsten stolpern lassen:

1. **(0,0) ist oben links.** Nicht unten links wie im Mathe-Unterricht.
2. **y wächst nach unten.** NORTH (nach oben) heißt y wird **kleiner**, SOUTH heißt y wird **größer**. Das fühlt sich falsch an — ist aber so.
3. **x ist die Spalte (links/rechts), y die Zeile (oben/unten).**

Kleiner Merksatz für die Richtungen:

NORTH	=	y - 1	(nach oben)
WEST	=	x - 1	EAST = x + 1
SOUTH	=	y + 1	(nach unten)

Die wichtigsten Werkzeuge

Für Distanz-, Richtungs- und Gegnersuche-Fragen müsst ihr nicht selbst rechnen — dafür gibt es fertige Toolkit-Funktionen (`import framework.arena.*` reicht, kein zusätzlicher Import-Pfad nötig). Volle Übersicht mit allen Funktionen: [toolkit-referenz.md](#).

“Wer ist der nächste Gegner?”

```
val ziel = sensors.nearestEnemy() ?: return Action.Wait
```

“Wie komme ich zu ihm hin, oder weg von ihm?”

```
val hinRichtung = self.approachDirectionTo(ziel.position)
val wegRichtung = self.fleeDirectionFrom(ziel.position)
```

“Steht er in meiner Schusslinie?”

```
if (sensors.canShoot(ziel)) { /* schießen */ }
```

Fast die gesamte Bewegungs- und Schuss-Logik im ganzen Backlog ist am Ende nur: *welche Toolkit-Funktion passt hier, und wie kombiniere ich sie mit einer if/else-Entscheidung?*

Epic 1 — Bewegung & Überleben

Story 1.1 — Zufällige Bewegung · 1 Punkt

Ziel: Bot bewegt sich jeden Tick in eine zufällige Richtung. Reiner “Läuft mein Bot überhaupt?”-Test.

Leitfragen: - Wie bekomme ich alle vier Richtungen als Liste? (Tipp: `Direction.entries`) - Wie wähle ich davon eine zufällig aus? (Tipp: `.random()`) - Was muss `decide()` am Ende zurückgeben? Eine `Action.Move(...)`.

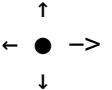
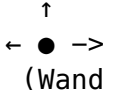
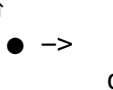
Taktik-Gedanke: Strategisch bringt Zufallslauf nichts — aber ihr seht sofort, ob euer Bot in der App auftaucht und sich bewegt. Genau darum geht's hier. Erst wenn das klappt, lohnen sich die schwereren Storys.

Häufiger Fehler: `Action.Move` ohne Richtung, oder `Direction.random()` statt `Direction.entries.random()`.

Story 1.2 — Am Rand bleiben / Zentrum meiden · 1 Punkt

Ziel: Bot soll sich nicht in der Mitte aufhalten, sondern Richtung Rand.

Warum ist das taktisch klug? In der Mitte kann euch aus **allen vier** Richtungen jemand treffen. Am Rand fällt schon eine Richtung weg, in der Ecke sogar zwei:

Mitte (schlecht):	Rand (besser):	Ecke (am sichersten):
		
4 Angriffs-seiten	3 Seiten (Wand unten)	nur 2 Seiten offen

Leitfragen: - Bin ich zu nah an der Mitte? Dafür gibt's schon eine fertige Funktion: `sensors.isNearCenter()`. - Wenn ja: in welche Richtung ist der nächste Rand? Auch fertig: `sensors.directionToNearestEdge()`.

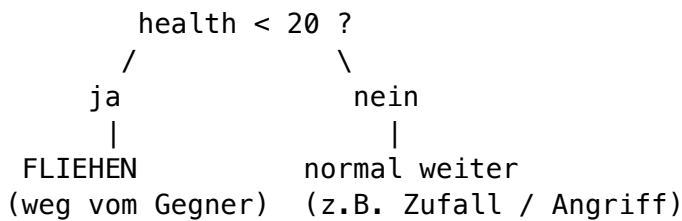
```
if (sensors.isNearCenter()) {  
    return Action.Move(sensors.directionToNearestEdge())  
}
```

Taktik-Gedanke: Mehr braucht die Story eigentlich nicht — die beiden Toolkit-Funktionen übernehmen komplett, was ihr sonst mit `abs()` und Vergleichen selbst hätten rechnen müssen. Wer mag, ergänzt noch ein `else`-Verhalten für den Fall, dass man schon am Rand steht (z.B. aus 1.1 oder Epic 2).

Story 1.3 — Flucht bei niedriger Gesundheit · 2 Punkte

Ziel: Sinkt `health` unter 20, läuft der Bot **weg** vom nächsten Gegner. Sonst verhält er sich normal (z.B. wie in Epic 2, oder erst mal Zufallslauf).

Das ist der erste “wenn ... dann anders”-Bot. Bisher hat euer Bot immer dasselbe gemacht. Jetzt baut ihr eine **Weiche**: je nach Situation zwei ganz verschiedene Verhalten. Genau dieses Muster braucht ihr später in Story 3.1 (Zustandsmaschine) wieder — nur mit mehr Zuständen.



Das Akzeptanzkriterium verlangt drei Dinge — hakt sie einzeln ab: 1. `health < 20` -> weg vom **nächsten** Gegner bewegen. 2. `health >= 20` -> normales Verhalten (irgendwas Sinnvolles, kein Absturz). 3. **kein** Gegner mehr da -> keine Exception, sondern `Action.Wait`.

Schritt 1 — nächsten Gegner finden (und den Leer-Fall abfangen) Fliehen könnt ihr nur **vor** jemandem. Also zuerst: wer ist der nächste Gegner? Der Leer-Fall ist keine Kür, sondern Pflicht (Kriterium 3) — ohne ihn stürzt der Bot ab, sobald der letzte Gegner tot ist und `sensors.others` leer wird. Beides übernimmt eine einzige Toolkit-Funktion:

```
val naechster = sensors.nearestEnemy() ?: return Action.Wait
```

`nearestEnemy()` gibt null zurück, wenn keiner mehr lebt — der `?:`-Fallback fängt genau das ab.

Schritt 2 — Fluchtrichtung bestimmen Auch das Vorzeichen-Dreh-Problem (weg statt hin, richtige Achse zuerst) nimmt euch eine Funktion ab:

```
val richtung = sensors.self.position.fleeDirectionFrom(naechster.position)
```

`fleeDirectionFrom` schaut sich an, auf welcher Achse der Abstand größer ist, und liefert die Richtung **weg** vom Gegner auf genau dieser Achse — ihr müsst weder `abs()` noch Vorzeichen selbst anfassen.

Zum Verständnis, was dahinter passiert: schaut euch die Beschreibung von `approachDirectionTo/fleeDirectionFrom` in [toolkit-referenz.md](#) an — dort steht auch, warum die Karte (y wächst nach **unten**) das Ergebnis beeinflusst.

Schritt 3 — der else-Zweig (genug HP) Kriterium 2: Bei `health >= 20` soll der Bot **normal** weitermachen. Was “normal” ist, hängt davon ab, wie weit ihr seid:

- Habt ihr Epic 2 noch nicht: einfach `Action.Move(Direction.entries.random())` oder mit `approachDirectionTo` auf den Gegner **zu** laufen.
- Habt ihr Epic 2 schon: hier eure Angriffs-Logik (schießen / verfolgen) einsetzen.

Zusammengebaut:

```
val naechster = sensors.nearestEnemy() ?: return Action.Wait

return if (sensors.self.health < 20) {
    Action.Move(sensors.self.position.fleeDirectionFrom(naechster.position)!!)
} else {
    // z.B. Angriffs-Logik aus Epic 2
    Action.Move(sensors.self.position.approachDirectionTo(naechster.position)!!)
}
```

fleeDirectionFrom/approachDirectionTo liefern nur dann null, wenn ihr exakt auf dem Gegner steht (kann in der Praxis nicht passieren, da Felder nicht doppelt belegt werden) — deshalb ist !! hier vertretbar.

Häufige Fehler bei genau dieser Story: - **nearestEnemy()**-Ergebnis nicht abgefangen -> Absturz, sobald der letzte Gegner stirbt. Der Bot wird dann von der Engine “eingefroren”. - **fleeDirectionFrom/approachDirectionTo** verwechselt -> Bot rennt beim Fliehen genau **auf** den Gegner zu. Der wichtigste Test: HP künstlich runter (z.B. gegen mehrere Bots), zuschauen — läuft er wirklich **weg**? - **health <= 20 statt < 20** -> Grenzfall; die Story sagt “unter 20”, also < 20. Kleinigkeit, aber im Review erwähnen.

Story 1.4 — Keine Bewegung verschwenden (Randcheck) · 1 Punkt

Ziel: Bevor der Bot sich bewegt, prüft er, ob das Zielfeld überhaupt in der Arena liegt.

Leitfragen: - Wie komme ich von “ich will nach NORTH” auf “welche Position wäre das dann”? -> `Position.moved(direction)` (aus `Models.kt`, kein Toolkit-Import nötig). - Wie prüfe ich, ob diese Position noch in der Arena liegt? -> `zielPosition.isInsideArena(sensors.arenaWidth, sensors.arenaHeight)`. - Was mache ich, wenn die gewählte Richtung ungültig wäre? -> eine andere Richtung suchen, die gültig ist (z.B. mit `Direction.entries.first { ... }`).

Taktik-Gedanke: Ohne diese Prüfung verschenkt ihr am Rand ständig Ticks — der Bot “will” gegen die Wand laufen, die Engine lässt die Bewegung stillschweigend ausfallen, und ihr steht einfach da. Mit dem Randcheck bewegt sich der Bot immer irgendwohin, auch am Rand.

Häufiger Fehler: Nur die **gewünschte** Richtung prüfen, aber beim “ungültig”-Fall vergessen, überhaupt eine Ersatzrichtung zurückzugeben.

Story 1.5 — Sicherheitsabstand zum nächsten Gegner halten · 2 Punkte

Ziel: Bot hält unabhängig von seiner HP einen Mindestabstand zum nächsten Gegner — unabhängig von Story 1.3 (dort geht’s um HP, hier nur um Distanz).

Leitfragen: - Wie messe ich den Abstand auf einem Raster ohne Diagonalen? -> `meinePosition.manhattanDistanceTo(gegner.position)`. - Ab welcher Distanz ist es “zu nah”? Das legt ihr selbst fest (z.B. < 2). - Was mache ich, wenn’s zu nah ist? -> wegbewegen, genau wie in 1.3 (`fleeDirectionFrom`).

```
val naechster = sensors.nearestEnemy() ?: return Action.Wait
val abstand = sensors.self.position.manhattanDistanceTo(naechster.position)
if (abstand < 2) {
    return Action.Move(sensors.self.position.fleeDirectionFrom(naechster.position)!!)
}
```

Taktik-Gedanke: Diese Story lohnt sich besonders, **bevor** ihr mit Epic 2 (Angriff) anfangt — ein Bot, der nie ungewollt direkt neben einem Gegner steht, ist später auch beim Schießen und Verfolgen robuster.

Story 1.6 – Fluchtrichtung mit Wandvermeidung kombinieren · 2 Punkte

Ziel: Kombiniert 1.3 (Flucht bei niedriger HP) mit 1.4 (Randcheck) — die Flucht darf nicht gegen die Wand laufen.

Leitfragen: - Was passiert, wenn `fleeDirectionFrom` eine Richtung liefert, die aus der Arena hinausführt? Testet das gezielt: Bot in eine Ecke stellen, HP unter 20, Gegner so platzieren, dass die “natürliche” Fluchtrichtung nach draußen zeigt. - Löst sich das mit genau demselben Muster wie in 1.4? Ja — erst `isInsideArena` prüfen, dann bei Bedarf eine andere gültige Richtung suchen.

```
val fluchtRichtung = sensors.self.position.fleeDirectionFrom(naechster.position)!!
val fluchtZiel = sensors.self.position.moved(fluchtRichtung)
if (!fluchtZiel.isInsideArena(sensors.arenaWidth, sensors.arenaHeight)) {
    // andere gültige Richtung suchen, siehe 1.4
}
```

Taktik-Gedanke: Baut wenn möglich direkt auf eurem 1.4-Code auf, statt den Randcheck ein zweites Mal neu zu schreiben — gute Übung, um zu sehen, wie sich kleine Bausteine kombinieren lassen.

Epic 2 – Angriff

Story 2.1 — Dauerfeuer in feste Richtung · 1 Punkt

Ziel: Bot schießt jeden Tick in dieselbe feste Richtung.

Leitfragen: - Welche Action schießt? -> Action.Shoot(Direction.EAST). - Warum trifft ihr damit fast nie? -> Ein Schuss trifft nur, wenn ein Gegner **exakt** in derselben Zeile (bei EAST/WEST) bzw. Spalte (bei NORTH/SOUTH) steht. Zufällig passt das selten.

Taktik-Gedanke: Testet gegen StillstandBot — der bewegt sich nicht, also könnt ihr euch so hinstellen, dass die Schüsse treffen. Diese Story ist nur zum Kennenlernen von Shoot; die “richtige” Zielerei kommt in 2.2.

Story 2.2 – Auf Gegner in Sichtlinie zielen · 2 Punkte

Ziel: Nur schießen, wenn ein Gegner **wirklich** in der Schusslinie steht – und dann in die richtige Richtung.

Was heißt “in Sichtlinie”? Auf dem Raster gibt es keine schrägen Schüsse. Ihr trefft jemanden nur, wenn er in derselben **Spalte** (gleiches x) oder derselben **Zeile** (gleiches y) steht:

G gleiches x wie ● → Schuss nach NORTH trifft G
 ● · · G gleiches y wie ● → Schuss nach EAST trifft den rechten G

G kann NICHT getroffen werden, wenn er weder in der Zeile noch in der Spalte steht (schräg):

- • G
- • • weder gleiches x noch gleiches y $\rightarrow$ kein Schuss

Leitfragen: - Wie prüfe ich, ob ein Gegner treffbar ist? -> fertige Funktion: `sensors.canShoot(gegner)`.
- Wie finde ich den nächsten treffbaren Gegner? -> `sensors.nearestEnemyInLineOfSight()`
übernimmt "erst filtern, dann nächsten nehmen" für euch komplett. - Wie leite ich aus der Position

die Schussrichtung ab? -> `sensors.self.position.directionTo(ziel.position)`. - Kein Gegner in Linie? -> **nicht** ins Leere schießen, lieber `Action.Wait` (oder in 2.3: hinlaufen).

```
val ziel = sensors.nearestEnemyInLineOfSight()
if (ziel != null) {
    return Action.Shoot(sensors.self.position.directionTo(ziel.position)!!)
}
return Action.Wait
```

`directionTo` liefert nur dann `null`, wenn die beiden Positionen weder gleiches `x` noch gleiches `y` haben — bei einem Ziel aus `nearestEnemyInLineOfSight()` ist das ausgeschlossen, deshalb ist `!!` hier sicher.

Häufiger Fehler: Erst den nächsten Gegner suchen und **dann** prüfen, ob er in Linie steht. `nearestEnemyInLineOfSight()` macht es automatisch richtig herum: **erst filtern** (wer ist überhaupt treffbar), **dann** den nächsten davon nehmen.

Story 2.3 — Gegner verfolgen · 2 Punkte

Ziel: Steht kein Gegner in Schusslinie -> einen Schritt auf den nächsten zulaufen. Kombiniert mit 2.2 ergibt das einen echten Jäger-Bot.

Die Gesamtlogik pro Tick:

```

    nächsten Gegner suchen
      |
    steht er in Linie (gleiches x oder y)?
      /      \
    ja        nein
    |          |
  SCHIESSEN   HINLAUFEN
(Richtung aus 2.2) (dx/dy -> Schritt zum Gegner)
```

Leitfragen: - “Hinlaufen” ist dasselbe wie “Fliehen” aus 1.3 — nur die passende Toolkit-Funktion ist `approachDirectionTo` statt `fleeDirectionFrom`: `sensors.self.position.approachDirectionTo(ziel.position)`.
- Warum nähert man sich damit “diagonal treppenförmig”? -> weil `approachDirectionTo` abwechselnd die Achse mit dem größeren Abstand nimmt, bis man in einer Linie steht.

```
val ziel = sensors.nearestEnemyInLineOfSight()
return if (ziel != null) {
    Action.Shoot(sensors.self.position.directionTo(ziel.position)!!)
} else {
    val naechster = sensors.nearestEnemy() ?: return Action.Wait
    Action.Move(sensors.self.position.approachDirectionTo(naechster.position)!!)
}
```

Taktik-Gedanke: Das ist genau der `ChaserBot` aus `bots/examples/`. Wenn ihr nicht weiterkommt: **nicht abschreiben**, aber die Struktur anschauen und verstehen, dann selbst nachbauen.

Story 2.4 — Nicht ins Leere schießen · 1 Punkt

Ziel: Vor jedem Schuss prüfen, ob wirklich ein Gegner getroffen werden kann.

Leitfragen: - Welche Funktion sagt euch, ob ein Ziel gerade treffbar ist? -> `sensors.canShoot(ziel)`.
- Was tut ihr, wenn kein Ziel treffbar ist? -> **nicht** schießen, sondern etwas anderes (bewegen, siehe

1.1 oder später 2.3).

Taktik-Gedanke: Diese Story ist eine bewusst kleinere Vorstufe zu 2.2 — hier reicht es, den nächsten Gegner zu nehmen und **vor** dem Schuss einmal `canShoot` zu prüfen. Der “beste” Gegner in Sichtlinie (2.2) kommt erst danach.

Story 2.5 — Schwächsten Gegner in Sichtlinie zuerst angreifen · 2 Punkte

Ziel: Stehen mehrere Gegner gleichzeitig in Schusslinie, wird der mit den wenigsten HP zuerst angegriffen.

Leitfragen: - Wie bekomme ich **alle** treffbaren Gegner, nicht nur den nächsten? -> `sensors.enemiesInLineOfSight()` liefert eine Liste. - Wie wähle ich daraus den mit der niedrigsten health? -> `minByOrNull { it.health }` auf dieser Liste. - Achtung: `sensors.weakestEnemy()` (aus Story 3.2) sucht den schwächsten **aller** Gegner, nicht nur der in Sichtlinie — für diese Story braucht ihr die Kombination aus beidem.

```
val ziel = sensors.enemiesInLineOfSight().minByOrNull { it.health }
```

Taktik-Gedanke: Ein Gegner mit wenig HP ist mit einem Schuss vielleicht schon erledigt — den zuerst auszuschalten reduziert die Zahl der Gegner schneller, als stur den nächsten zu bekämpfen.

Story 2.6 — Rückzug beim Schießen vermeiden · 2 Punkte

Ziel: Steht der Bot in Schusslinie, aber zu nah dran, weicht er zurück statt weiter anzugreifen.

Leitfragen: - Wie kombiniere ich “kann ich schießen” mit “ist es zu nah”? -> erst `sensors.canShoot(ziel)` prüfen, dann **innerhalb** davon den Abstand mit `manhattanDistanceTo` checken. - Was mache ich bei “zu nah”? -> `fleeDirectionFrom`, wie schon in 1.3/1.5 — gleicher Baustein, neue Situation.

Taktik-Gedanke: Ohne diese Story bleibt euer Verfolger-Bot (2.3) am Ende direkt neben dem Gegner stehen, sobald er ihn treffen kann — unnötig riskant, da der Gegner dann auch euch leicht trifft. Ein kleiner Sicherheitsabstand beim Schießen macht den Bot deutlich überlebensfähiger.

Epic 3 — Strategie & Zustände

Story 3.1 — Zustandsmaschine (Patrouille / Angriff / Flucht) · 5 Punkte

Ziel: Bot hat drei klar benannte Zustände und wechselt je nach Lage.

Das ist keine **neue** Logik — es ist das **Aufräumen** von allem aus Epic 1 & 2 in drei saubere Schubladen. Denkt an eine Ampel: je nach Situation ein anderer Zustand, jeder Zustand macht etwas klar Unterscheidbares.

health < 20 → FLUCHT (Logik aus Story 1.3)
und Gegner da?

Gegner da (aber genug HP) → ANGRIFF (Logik aus Story 2.2 + 2.3)

kein Gegner → PATROUILLE (Logik aus Story 1.1)

Leitfragen: - Wie stelle ich drei Zustände dar? -> am saubersten mit `enum class BotState { PATROUILLE, ANGRIFF, FLUCHT }`. - Wie bestimme ich den Zustand? -> mit einem `when { ... }`,

das die aktuelle Lage prüft. **Reihenfolge zählt:** zuerst FLUCHT prüfen, dann ANGRIFF, dann Rest. - Warum zuerst Flucht? -> Ein fast toter Bot soll fliehen, **auch wenn** ein Gegner in Schussweite steht. Prüft ihr Angriff zuerst, stirbt er beim Angreifen.

Taktik-Gedanke: Muss der Zustand gespeichert werden? Nein — es reicht, ihn **jeden Tick neu** aus sensors zu berechnen. Das macht den Bot robust: er reagiert immer auf die aktuelle Lage.

Story 3.2 — Zielpriorisierung bei mehreren Gegnern · 2 Punkte

Ziel: Bei mehreren Gegnern nicht wahllos den erstbesten angreifen, sondern nach einer klaren Regel auswählen.

Der ganze Trick ist eine andere Toolkit-Funktion. Bisher habt ihr immer den *nächsten* Gegner gesucht:

```
val ziel = sensors.nearestEnemy() ?: return Action.Wait
```

Jetzt nehmt ihr statt nearestEnemy() einfach weakestEnemy() — sucht nach einem anderen Kriterium (wenigste HP statt kleinster Abstand), aber liefert euch genauso ein einzelnes Ziel (oder null, wenn keiner mehr lebt):

```
val ziel = sensors.weakestEnemy() ?: return Action.Wait
```

Das ist der ganze Lernpunkt der Story: **welches Kriterium** passt zu meiner Taktik?

Leitfragen: - Welche Regel wollt ihr? Mögliche Kriterien: - **schwächster zuerst** (sensors.weakestEnemy()) — schaltet ihr am schnellsten aus einem Kampf aus. - **nächster zuerst** (sensors.nearestEnemy()) — am leichtesten zu erreichen/treffen. - Egal welches — schreibt als Kommentar dazu, **warum** ihr es gewählt habt. - Der Rest (schießen/hinlaufen) bleibt exakt wie in Epic 2, nur mit ziel aus weakestEnemy() statt nearestEnemy(). Achtung: für den Schuss müsst ihr trotzdem mit sensors.canShoot(ziel) prüfen, ob das gewählte Ziel gerade in Sichtlinie steht — das schwächste Ziel ist nicht automatisch auch treffbar.

Häufiger Fehler: Nur das Kriterium ändern, aber vergessen, danach überhaupt zu schießen/laufen. Denkt dran, die Aktion aus 2.2/2.3 anzuhängen.

Story 3.3 — Kür-Aufgabe (freie Idee) · 5 Punkte

Ziel: Eigene Idee, die über die Vorgaben hinausgeht. Ihr formuliert die Story selbst, der Dozent bestätigt kurz die Machbarkeit.

Was geht gut mit der API? - **An der Wand entlang** patrouillieren (Position mit arenaWidth/arenaHeight vergleichen). - **Gegner in die Ecke drängen** (Bewegung relativ zu mehreren others). - **“Nur schießen, wenn sicher”** — z.B. nur feuern, wenn genau **ein** Gegner in Linie steht, nicht mehrere. - **Gegner-Zug merken** — mit einer var-Property speichert ihr die letzte Position eines Gegners und schätzt, wohin er läuft. (Vorher mit Dozent kurz besprechen.)

Was geht NICHT? - Ihr könnt **nicht in die Zukunft sehen**. sensors zeigt nur den Zustand **jetzt**, bevor sich alle bewegen. Ein Schuss auf ein laufendes Ziel ist immer eine **Wette**, kein sicherer Treffer. - Ihr könnt nicht sehen, was ein Gegner als Nächstes *entscheidet* — nur, wo er gerade steht.

Taktik-Gedanke: Fangt klein an. Eine simple, funktionierende Idee ist mehr wert als eine geniale, die nicht läuft. Baut auf eurem Epic-3.1-Bot auf und fügt **einen** cleveren Zusatz hinzu.

Story 3.4 — Zeitgesteuerte Startphase (Patrouille zuerst) · 2 Punkte

Ziel: In den ersten Ticks patrouilliert der Bot nur, statt sofort in einen Kampf zu laufen.

Leitfragen: - Woher weiß der Bot, wie viele Ticks das Match schon läuft? -> `sensors.tick` (zählt bei jedem `decide()`-Aufruf hoch). - Wie legt ihr den Schwellenwert fest, ab dem "richtig" gespielt wird? -> als klar benannte Konstante im Code, z.B. `const val PATROUILLE_TICKS = 10`.

```
if (sensors.tick < PATROUILLE_TICKS) {  
    return Action.Move(Direction.entries.random())  
}
```

Taktik-Gedanke: Eine kurze "Beobachtungsphase" am Matchbeginn verhindert, dass euer Bot sofort unüberlegt auf den erstbesten Gegner zurennt, bevor die Lage überhaupt klar ist.

Story 3.5 — Abklingzeit nach der Flucht · 3 Punkte

Ziel: Nach einer Flucht bleibt der Bot noch einige Ticks vorsichtig, statt sofort wieder anzugreifen.

Das ist die erste Story, bei der ihr euch wirklich etwas über mehrere Ticks hinweg merken müsst — nicht mehr "jeden Tick neu berechnen" wie in 3.1, sondern ein Zähler, der zwischen den `decide()`-Aufrufen erhalten bleibt.

Leitfragen: - Wie merkt sich der Bot etwas zwischen zwei `decide()`-Aufrufen? -> eine var-Property auf Klassenebene (nicht lokal in `decide()`!). - Wann wird der Zähler zurückgesetzt, wann hochgezählt? - Fliehe ich gerade (`health < 20`)? -> Zähler auf 0. - Fliehe ich nicht? -> Zähler hochzählen. - Wie lange bleibt der Bot "vorsichtig"? Selbst festgelegte Anzahl Ticks (z.B. 5) nach dem letzten Flucht-Tick.

```
var ticksSeitFlucht = ABKLINGZEIT_TICKS // Start: "nicht mehr vorsichtig"  
  
// in decide():  
if (sensors.self.health < 20) {  
    ticksSeitFlucht = 0  
} else {  
    ticksSeitFlucht++  
}  
  
val nochVorsichtig = ticksSeitFlucht < ABKLINGZEIT_TICKS
```

Häufiger Fehler: Den Zähler als lokale Variable **innerhalb** von `decide()` deklarieren — dann vergisst der Bot bei jedem Aufruf wieder alles. Er muss eine Property der Bot-Klasse sein.

Taktik-Gedanke: Guter Anlass, den Unterschied zwischen zustandslos (3.1: alles aus `sensors` neu berechnet) und zustandsbehaftet (hier: `var` merkt sich etwas) am eigenen Bot zu erleben.

Epic 4 — Qualität (optional)

Story 4.1 — Unit-Test für eure Logik · 2 Punkte

Ziel: Ein Test, der `decide()` mit selbst gebautem `Sensors` aufruft und das Ergebnis prüft.

Warum geht das so einfach? `decide()` ist eine **reine Funktion**: rein `Sensors`, raus `Action`. Ihr braucht keine laufende App — ihr baut einfach von Hand eine Situation und prüft, was rauskommt.

Leitfragen: - Wie baue ich eine Test-Situation? -> `RobotState(...)` und `Sensors(...)` sind normale data class-Objekte, die ihr direkt erzeugen könnt. - Welche Situation teste ich? -> z.B.

Gegner steht östlich in gleicher Zeile -> erwartet: `Action.Shoot(Direction.EAST)`. - Wo muss die Datei liegen? -> `src/test/kotlin/bots/teamX/...` (Ordner ggf. neu anlegen), package muss zum Pfad passen.

Taktik-Gedanke: Testet einen Fall, bei dem ihr die Antwort **sicher** wisst (z.B. Gegner exakt rechts). Dann seht ihr sofort, ob eure Richtungslogik stimmt.

Story 4.2 — Testduell protokollieren · 1 Punkt

Kein Code. App starten, euren Bot + einen Beispiel-Bot wählen, Match laufen lassen, Ergebnis in 1-2 Sätzen notieren.

Epic 5 — Turniervorbereitung

Story 5.1 — Finale Version festlegen · 1 Punkt

Stellt sicher, dass in `teamXBots = listOf(...)` **genau** die Bot-Version steht, die antreten soll — nicht drei halbfertige Zwischenstände. `./gradlew build` muss grün sein.

Story 5.2 — Strategie in 1-2 Sätzen · 1 Punkt

Könnt ihr in eigenen Worten sagen, **wann** euer Bot angreift, flieht, patrouilliert? Wenn ja: fertig. Wenn nicht: das ist ein Zeichen, dass euer Bot vielleicht noch zu unübersichtlich ist — gute Gelegenheit zum Aufräumen.

Wenn ihr komplett feststeckt

1. **Läuft der Bot überhaupt?** Zurück zu Story 1.1 — taucht er in der Auswahl auf, bewegt er sich?
2. **Absturz?** Meist `sensors.others` leer und trotzdem `sensors.others[0]` oder `.first()` benutzt. Immer **vorher** mit `?:` gegen null von `sensors.nearestEnemy()` (oder `sensors.others.isEmpty()`) abfangen.
3. **Läuft/schießt falsch herum?** `fleeDirectionFrom` und `approachDirectionTo` verwechselt? Dran denken: y wächst nach **unten**.
4. **Schießt nie?** Steht ein Gegner wirklich in gleicher Zeile/Spalte? Prüft mit `sensors.canShoot(gegner)`. Testet erst gegen `StillstandBot`, da ist es kontrollierbar.
5. Schaut euch die fertigen Bots in `bots/examples/` an — **nicht abschreiben**, sondern verstehen und selbst nachbauen.